Harnessing Functional Programming for Parallelism

Aaditya Joshi (IMT2022092)

Krish Dave (IMT2022043)

Motivation

Why Study Parallelism?

Modern processors contain multiple CPU cores, making parallel computation essential for improving performance.

However, writing parallel programs is difficult because of:

- Race conditions
- Shared mutable state
- Synchronization overhead
- Deadlocks and non-deterministic bugs

Main Question

Can functional programming simplify parallel programming while still achieving satisfactory performance?

Project Objective

Goals of the Project

We implemented the same parallel algorithms in:

- Haskell (functional paradigm)
- C++ (low-level imperative paradigm)
- Python (high-level imperative paradigm)

We Compared

- Execution speed
- Scalability
- Parallel efficiency
- Ease of implementation

Focus

Understanding how different programming paradigms express and manage parallelism.

Functional Programming & Parallelism

Why Functional Programming Helps

Functional programming naturally supports parallel execution because of:

Immutability: Data is not modified after creation, eliminating most shared-state issues.

Referential Transparency: Functions always produce the same output for the same input, making computations predictable.

Higher-Order Functions: Operations like map, fold, and filter naturally expose independent computations.

Separation of Logic and Execution: The algorithm can remain unchanged while the parallel execution strategy is modified independently.

Haskell Parallelism Features Used

Haskell Constructs Used

Eval Monad: Provides fine-grained control over parallel task evaluation.

Par Monad: Supports deterministic parallel computation without race conditions.

Strategies: Separates algorithm logic from execution strategy.

Async: Implements futures/promises style concurrency.

STM (Software Transactional Memory): Allows safe shared-state concurrency using transactions instead of locks.

Project Architecture

System Design

The project was organized as a modular benchmarking framework.

Components

- Sequential and parallel implementations
- Benchmarking utilities
- Independent benchmark drivers
- Result collection and analysis

Important Design Principle

Separation of Algorithm and Parallel Strategy

This allowed:

- Reusing the same algorithm logic
- Experimenting with different parallel execution methods
- Cleaner implementation and benchmarking

Algorithms Implemented

Each problem represents a different style of parallel computation.

1) Parallel Merge Sort
Task Parallelism

2) Matrix Multiplication
Data Parallelism

3) MapReduce Word Count
Higher-Order Parallelism

4) Monte Carlo Pi Estimation
Probabilistic Parallel

5) K-Means Clustering
Iterative Parallelism

6) Trapezoidal Rule
Domain Decomposition

Parallel Merge Sort

Merge Sort — Task Parallelism

Approach: The input list is recursively divided; left and right halves are sorted in parallel, then merged.

Haskell uses: Eval Monad · rpar · rseq · force

Key Insight: Depth control was necessary to avoid creating too many small parallel tasks.

Observation

- Haskell achieved good speedup with minimal code changes.
- C++ achieved the best overall performance.
- Python multiprocessing overhead reduced scalability for smaller inputs.

Input Size	Haskell Seq	Haskell Par	C++ Seq	C++ Par	Python Seq	Python Par
10K	14.35 ms	3.76 ms (3.81x)	0.64 ms	0.27 ms (2.37x)	23.12 ms	114.30 ms (0.20x)
50K	68.13 ms	31.97 ms (2.13x)	3.2 ms	0.8 ms (4.00x)	138.32 ms	123.61 ms (1.12x)
100K	172.83 ms	76.69 ms (2.25x)	7.47 ms	2.05 ms (3.64x)	313.87 ms	278.82 ms (1.13x)

Matrix Multiplication

Matrix Multiplication — Data Parallelism

Approach: Each row of the output matrix can be computed independently.
Haskell uses: parMap · chunk-based scheduling · evaluation strategies
Key Optimization: Chunking reduced scheduling overhead by grouping computations together.

Observation

- C++ showed strong scaling for larger matrices.
- Haskell achieved moderate speedups.
- Python remained significantly slower due to multiprocessing overhead.

Size	Haskell Seq	Haskell Par	C++ Seq	C++ Par	Python Seq	Python Par
64x64	3.12 ms	2.04 ms (1.53x)	56 us	78 us (0.72x)	27.33 ms	78.53 ms (0.35x)
128x128	18.77 ms	14.48 ms (1.30x)	920 us	232 us (3.97x)	178.95 ms	128.25 ms (1.40x)

MapReduce Word Count

Approach

- Split input text into chunks
- Count words independently (Map phase)
- Merge intermediate counts (Reduce phase)

Haskell uses: Par Monad for deterministic parallel execution.

Main Challenge: The merge/reduction phase introduced synchronization and communication overhead.

Observation & Insight

- C++ handled reduction efficiently.
- Haskell parallel execution was often slower than sequential.
- Aggregation-heavy workloads are harder to parallelize efficiently.

Words	Haskell Seq	Haskell Par	C++ Seq	C++ Par	Python Seq	Python Par
155K	85.15 ms	172.56 ms (0.49x)	12.73 ms	4.19 ms (3.04x)	149.52 ms	165.20 ms (0.91x)
465K	255.46 ms	450.32 ms (0.57x)	38.76 ms	11.45 ms (3.39x)	441.76 ms	181.71 ms (2.43x)

Monte Carlo Pi Estimation

Approach

- Random points are generated independently.
- Each worker computes local hit counts.
- Results are combined to estimate π.

Haskell uses: Async-based parallelism · STM-based shared-state concurrency.

Why It Parallelizes Well: Each simulation is independent and requires almost no communication.

Observation

- C++ achieved near-linear scaling.
- Haskell performed well until task overhead became too high.
- Python improved significantly for large workloads.

Samples	Haskell Seq	Haskell Par	C++ Seq	C++ Par	Python Seq	Python Par
100K	1.10 ms	0.40 ms (2.6x)	1.50 ms	366 us (4.11x)	20.24 ms	75.10 ms (0.27x)
1M	10.5 ms	3.5 ms (3.0x)	12.25 ms	3.04 ms (4.03x)	203.42 ms	121.07 ms (1.68x)
10M	163 ms	185.94 ms (0.88x)	122.72 ms	60.24 ms (2.04x)	2.08 s	660.33 ms (3.15x)

K-Means Clustering

Approach

The assignment phase was parallelized:

- Each point independently finds its nearest centroid.
  Main Challenge: Centroid recomputation requires synchronization after every iteration.

Observation

- Medium-sized datasets showed improvement.
- Large datasets suffered from synchronization overhead.
- Python performed poorly overall due to multiprocessing costs.
  Insight: Iterative algorithms are difficult to scale because of repeated synchronization barriers.

Points	Haskell Seq	Haskell Par	C++ Seq	C++ Par	Python Seq	Python Par
10K	64.05 ms	68.41 ms (0.94x)	17.40 ms	15.86 ms (1.10x)	273.69 ms	1.74 s (0.16x)
50K	772.82 ms	431.82 ms (1.79x)	15.34 ms	10.14 ms (1.51x)	1.90 s	3.23 s (0.59x)
100K	1.29 s	1.49 s (0.87x)	22.82 ms	49.01 ms (0.47x)	2.38 s	2.27 s (1.05x)

Trapezoidal Rule

Approach

- The integration interval is divided into independent subranges.
- Each worker computes its own portion of the integral.
- Partial results are summed together.

  Why It Works Well: Subtasks are completely independent and require no communication.

Observation

- Excellent scalability across all languages.
- Strong speedups in Haskell and C++.
- Python benefited significantly because computation outweighed overhead.

  Insight: Embarrassingly parallel workloads achieve the best scalability.

Function	N	Haskell Seq	Haskell Par	C++ Seq	C++ Par	Python Seq	Python Par
sin(x)	10M	94.88 ms	85.96 ms (1.10x)	46.52 ms	12.09 ms (3.85x)	1.47 s	436.29 ms (3.38x)
4/(1+x^2)	10M	73.43 ms	38.32 ms (1.92x)	13.90 ms	4.07 ms (3.41x)	1.96 s	449.63 ms (4.37x)
x^2	10M	99.24 ms	55.55 ms (1.79x)	13.88 ms	3.59 ms (3.87x)	1.38 s	455.29 ms (3.04x)

Benchmarking Framework

Framework Features

- Dedicated benchmark drivers for each algorithm
- Common timing utilities
- Sequential vs parallel comparison
- Configurable chunk sizes, thread counts, and worker counts

Important Detail

Full evaluation was forced in Haskell to avoid inaccurate measurements caused by lazy evaluation.

Purpose

Ensure fair and reproducible comparisons across all languages.

Overall Comparative Analysis

Haskell

- Cleaner and safer parallel abstractions
- Moderate runtime overhead
- Easier reasoning about concurrency

C++

- Highest raw performance
- Excellent scalability
- More implementation complexity

Python

- Simpler implementation
- High multiprocessing and serialization overhead
- Limited scalability for smaller workloads

General Trend: *Algorithms with minimal communication scaled the best across all languages and input sizes.*

Challenges Faced

Functional Programming Challenges

- Managing lazy evaluation
- Preventing excessive spark creation
- Choosing proper task granularity

Parallel Computing Challenges

- Synchronization overhead
- Load balancing
- Communication cost

Python-Specific Challenges

- Serialization overhead
- Process creation cost

Future Work

Distributed Parallelism

- Apache Spark
- MPI (Message Passing Interface)

GPU Acceleration

- CUDA/OpenCL-based implementations

Larger Real-World Datasets

- Scientific computing
- Machine learning workloads

Hybrid Models Combining

- Functional abstractions
- Low-level imperative optimizations

Conclusion

Main Findings

- Functional programming simplifies safe parallelism.
- Haskell enables expressive parallel abstractions with relatively small code changes.
- C++ provides the best raw performance and scalability.
- Python is easier to implement but suffers from high multiprocessing overhead.

Final Insight

The effectiveness of parallelism depends heavily on:

- Workload structure
- Communication requirements
- Task granularity

Thank You

Aaditya Joshi (IMT2022092) · Krish Dave (IMT2022043)